

Secret Leakage Monitor for Public Student Repositories

Real-time detection of hardcoded API keys and tokens using pattern matching and Shannon entropy, with instant alerts and a monitoring dashboard

Codefiesta 5.0 Flagship Hackathon (GIT Jaipur) | Cybersecurity Track | Problem Statement 03

Team: _____ | Date: 8 October 2026

Abstract

Students routinely commit hardcoded API keys, cloud credentials and tokens to public GitHub repositories. Automated scanners operated by attackers can discover such secrets within minutes, while the student usually learns about the leak only after misuse, for example an unexpected cloud bill or exhausted API credits. This paper presents the design of a lightweight, web-based Secret Leakage Monitor aimed at students, coding clubs and colleges. The system watches selected public repositories, fetches the added lines of each new commit, and scans them with a two-stage detection engine: provider-specific regular expressions for high-confidence matches, followed by a Shannon-entropy heuristic with contextual hints for unknown secret formats. Each finding is assigned a severity, stored only in masked form, and pushed within seconds to a Discord or Slack channel together with plain-language remediation steps. A React dashboard presents the detection history. The paper describes the problem, related work, architecture, algorithms, data model, security considerations, an implementation plan for a 24-hour build, an evaluation plan and known limitations.

Keywords: secret detection, credential leakage, GitHub, Shannon entropy, regular expressions, DevSecOps, student security, alerting.

1. Introduction

Version control platforms have made collaboration easy, but they have also made accidental disclosure easy. A beginner who is trying to make a project work quickly will often paste a key straight into the source code. Once the code is pushed to a public repository, the secret is visible to everyone, including bots that continuously crawl public commits looking for credential patterns.

Two properties make this problem especially harmful for students. First, students typically hold personal cloud or AI accounts linked to a payment method, so abuse translates directly into financial loss. Second, students are less likely to know that Git preserves history: deleting a key in a later commit does not remove it from earlier commits, so the secret remains exposed until it is revoked.

The central issue is the **detection delay**, the gap between the moment a secret becomes public and the moment its owner finds out. This project aims to shrink that gap to seconds and to tell the owner clearly what to do next.

2. Problem Statement and Objectives

The official problem statement (Cybersecurity #03) reads, in essence: students push hardcoded API keys and tokens to public repositories, which are discovered and exploited by automated scanners within minutes. The stated requirements are listed below with the component of our design that satisfies each.

Requirement	Our design component
GitHub webhook or API polling for new commits	Poller service (polling is primary, webhook is an optional extension)
Secret detection via regex patterns plus a Shannon entropy threshold	Two-stage Detector engine (Section 6)
Alert pipeline to Discord or Slack with repo, commit, file and secret type	Notifier service using incoming webhooks (Section 8)
Dashboard with detection history; demo fires an alert within seconds of a test commit	React dashboard and REST API (Sections 9 and 10); end-to-end latency target under 30 seconds

2.1 Objectives

- Detect secrets in newly pushed commits of monitored public repositories with high precision for well-known providers.
- Keep false positives manageable through severity levels, allowlists and contextual checks.
- Deliver an alert with actionable, plain-language guidance within seconds.
- Never store or display a full secret; the monitor itself must not become a new leak.
- Be buildable and demonstrable within a 24-hour hackathon using FastAPI, PostgreSQL and React.

2.2 Non-goals

- Scanning the full history of every repository (heavy; proposed only as future work).
- Automatic revocation of keys (each provider has its own API and authentication requirements).
- Machine-learning-based detection (unnecessary for the MVP and outside the team's current scope).

3. Background

3.1 What is a secret?

A secret is any value that grants access to a resource: cloud access keys, API keys, OAuth or personal access tokens, database passwords and connection strings, private keys, bot tokens and webhook URLs. Possession of the value is usually enough to act as the owner, which is why it behaves like a password.

3.2 How secrets leak through Git

- **Hardcoding:** the key is written directly in source code or configuration files.
- **Committed environment files:** a .env file is committed because it was not listed in .gitignore.
- **History persistence:** removing the secret in a later commit leaves it in earlier commits, so the repository remains compromised until the secret is revoked.
- **Forks and clones:** once copied by others, the secret cannot be recalled.

3.3 Attack timeline

Attackers run automated crawlers over the public commit stream. When a pattern such as a cloud access-key prefix is matched, the key is tested and, if valid, abused (for instance to launch compute resources, call paid APIs or read data). The defender's only effective window is the time before the crawler acts, which motivates a push-time or near-real-time alert.

4. Related Work and Positioning

Secret detection is an established area, and this project does not claim to invent it. Understanding existing tools is essential for positioning our contribution honestly.

Tool	What it does	Typical audience / limitation relevant to us
GitHub secret scanning and push protection	Scans public repositories automatically at no cost; push protection can block a push that contains a supported secret. Detections on public repositories are also reported to the issuing provider if it participates in the partner program.	Alerts for the repository owner appear in the repository's security area, so a student who does not look there may miss them. Detection relies on supported patterns.
GitHub public monitoring	Scans public repositories across GitHub for secrets associated with an enterprise's members or verified email domain.	Offered as part of enterprise-level plans, so it is not available to a typical college club.
GitGuardian	Commercial platform with real-time monitoring, many source-control systems, CI and incident workflows.	Aimed at companies; richer than needed for a student community and not free at scale.
TruffleHog, Gitleaks, detect-secrets	Open-source scanners using pattern rules, entropy and, in some cases, secret verification.	Command-line tools for developers; they provide no student-oriented alerting or club-level dashboard out of the box.

4.1 Gap addressed

Existing tools are strong detectors, but they are designed for developers and organizations. We identify three gaps that a student-focused monitor can fill: (1) **delivery**, meaning alerts in a channel students actually read, such as Discord; (2) **guidance**, meaning simple instructions on what to do right now, including the fact that deletion is not enough; and (3) **community view**, meaning a single dashboard through which a club or faculty can see exposure across many student repositories. Our contribution is therefore an integration and usability contribution built on well-known detection techniques, not a new detection algorithm.

5. Proposed System

The Secret Leakage Monitor is a web application consisting of a FastAPI backend, a PostgreSQL database, a React dashboard and a Discord/Slack notifier. Users register the repositories (or GitHub usernames) to monitor. The backend polls GitHub for new commits, extracts added lines, runs the detector, stores masked findings and sends alerts.

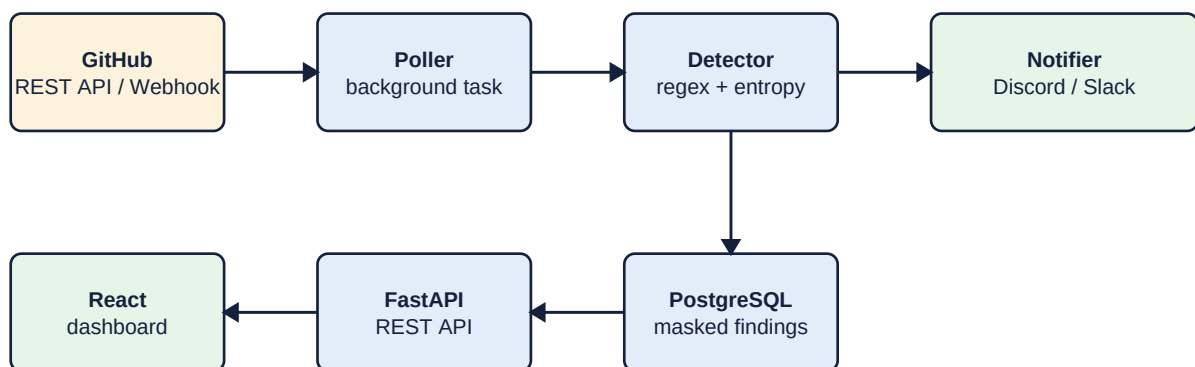


Figure 1. High-level architecture of the proposed monitor

5.1 End-to-end workflow

- **Watch:** every 10 to 15 seconds the poller asks GitHub whether a monitored repository has new commits (using conditional requests to save rate limit).

- **Fetch:** for each new commit the poller retrieves the changed files and their patch (diff).
- **Detect:** only lines beginning with a plus sign (additions) are scanned by the regex stage and the entropy stage.
- **Classify:** each match receives a secret type, a severity and a confidence.
- **Alert:** the notifier posts a message with repository, commit, file, line number, type, masked value and remediation steps.
- **Record:** the finding is saved in masked form and shown on the dashboard.

6. Detection Engine

Detection is split into two stages so that high-confidence matches are never diluted by noisier heuristics.

6.1 Stage 1: provider-specific regular expressions

Many providers issue credentials with recognisable prefixes and fixed lengths. Matching these formats gives high precision. The table lists example patterns; exact formats can change, so each pattern must be verified against the provider's current documentation before release.

Secret type	Example pattern (simplified)	Severity
AWS access key ID	AKIA[0-9A-Z]{16}	Critical
GitHub personal access token (classic)	ghp_[A-Za-z0-9]{36}	Critical
GitHub fine-grained token	github_pat_[A-Za-z0-9]{50,}	Critical
Stripe live secret key	sk_live_[0-9A-Za-z]{24,}	Critical
Private key block	-----BEGIN (RSA EC OPENSSH)?PRIVATE KEY-----	Critical
OpenAI-style API key	sk-[A-Za-z0-9_]{20,}	High
Google API key	Alza[0-9A-Za-z_]{35}	High
Slack token	xox[baprs]-[0-9A-Za-z-]{10,}	High
Discord webhook URL	https://discord(app)?.com/api/webhooks/[0-9]+/[w-]+	Medium
Database URL with password	(postgres mysql mongodb)(\+srv)?://[^\s]+:[^\s]+@	High
JSON Web Token	eyJ[A-Za-z0-9_]+\.[eyJ[A-Za-z0-9_]+\.[A-Za-z0-9_]+\.	Medium

6.2 Stage 2: Shannon entropy

For secrets whose format is unknown, we measure how random a candidate string looks. For a string s over an alphabet, the Shannon entropy in bits per character is $H(s) = - \sum \text{over each distinct character } c \text{ of } p(c) \times \log_2 p(c)$, where $p(c)$ is the relative frequency of c in s . Human-written identifiers have low entropy, whereas machine-generated keys have high entropy.

Entropy is applied only to candidate tokens: strings of at least 20 characters drawn from the base64 or hexadecimal alphabets and appearing in quotes or after an assignment operator. Typical starting thresholds used by open-source scanners are about 4.5 bits per character for base64-like strings and about 3.0 for hexadecimal strings. These are starting values that must be tuned on our own test data.

6.3 Context hints

The name of the variable on the left-hand side raises or lowers confidence. Names such as `api_key`, `secret`, `token`, `password`, `passwd`, `auth`, `credential` and `private_key` increase confidence; names such as `hash`, `checksum`, `uuid`, `sha`, `id` and `version` decrease it. An entropy-only match whose variable name has no sensitive hint is reported as Low severity.

6.4 False-positive control

- Ignore obvious placeholders (for example values containing example, dummy, changeme, your_key_here or only repeated characters).
- Downgrade matches in documentation and test paths (.md, test_, tests/, examples/, fixtures/).
- Skip binary files, lock files and minified bundles.
- Skip well-known non-secret random strings such as UUIDs and Git commit hashes.
- Support a per-repository allowlist and a user action to mark a finding as a false positive.

6.5 Severity model

Severity	Criteria	Example
Critical	Regex match for a credential that can directly cause financial loss or account takeover	AWS access key, private key, live payment key
High	Regex match for a service credential or a connection string with embedded password	OpenAI-style key, Slack token, database URL
Medium	Regex match for a limited-scope secret	Webhook URL, JWT
Low	Entropy-only match with a weak or no context hint	Random string assigned to an unclear variable

6.6 Core algorithm (pseudocode)

```
for each monitored repo:
    commits = github.list_new_commits(repo, since=last_seen_sha) # conditional request (ETag)
    for commit in commits:
        for file in github.get_commit_files(repo, commit.sha):
            if should_skip(file.path): continue
            for line_no, line in added_lines(file.patch):
                for rule in REGEX_RULES:
                    if m := rule.pattern.search(line):
                        emit(repo, commit, file, line_no,
                            rule.type, rule.severity, mask(m.group()))
                for token in candidate_tokens(line):
                    if looks_like_placeholder(token): continue
                    if (entropy(token) >= threshold_for(token)
                        and not already_reported(line_no)):
                        sev = "Medium" if has_sensitive_name(line) else "Low"
                        emit(repo, commit, file, line_no,
                            "high-entropy-string", sev, mask(token))
            last_seen_sha = commit.sha
```

The masking function keeps only a short prefix and suffix, for example AKIA*****MPLE. The full secret is processed in memory only for the duration of the scan and is never written to the database, logs or alerts.

7. GitHub Integration

7.1 Polling (primary)

Polling uses the GitHub REST API: listing commits for a repository, then fetching a single commit to obtain the per-file patch. Conditional requests with ETag or If-None-Match headers let unchanged repositories return a cheap not-modified response. Unauthenticated access is limited to a small hourly quota, whereas an authenticated personal access token provides a much larger quota, so a token with read access to public data is configured through an environment variable.

7.2 Webhook (optional extension)

For a push event, GitHub can send a payload to a public URL of our server, which removes the polling delay. Because a webhook needs a reachable address (for example through a tunnelling tool) and venue networks

can be unreliable, we treat the webhook as a bonus and keep polling as the dependable default. Each webhook request should be verified using the shared-secret signature header before processing.

7.3 Practical limits

- Very large diffs may omit the patch field; such files are flagged as not fully scanned.
- The number of monitored repositories is bounded by the polling interval and the API quota, so the interval can be adapted per repository.
- Only public repositories are monitored; no private data is accessed.

8. Alert Pipeline

Alerts are delivered through an incoming-webhook URL for a Discord (or Slack) channel. The webhook URL is itself a secret and is stored in an environment variable, never in the repository. Each message contains the following fields.

Field	Content
Title and severity	Colour-coded embed, for example CRITICAL: AWS access key exposed
Repository and commit	Repository name, short commit SHA and a link to the commit
File and line	File path and line number of the finding
Secret type and masked value	Type label and masked preview such as AKIA*****MPLE
What to do now	Short numbered steps (see below)
Detected at	Timestamp and latency from commit time

8.1 Remediation text

Every alert states that deleting the line is not sufficient. The standard guidance is: (1) revoke or rotate the key in the provider's console immediately; (2) generate a new key and store it in a .env file; (3) add .env to .gitignore; (4) check the provider's usage and billing logs for unauthorised activity; (5) optionally remove the secret from history using a history-rewriting tool, understanding that revocation is still mandatory. Provider-specific links (for example the AWS or OpenAI key pages) are included per secret type.

9. Data Model

PostgreSQL stores monitored repositories, scan checkpoints and masked findings. No raw secret is ever stored.

Table	Main columns
repositories	id, owner, name, added_by, interval_seconds, last_seen_sha, etag, is_active, created_at
findings	id, repo_id, commit_sha, file_path, line_no, secret_type, severity, confidence, masked_value, fingerprint (hash), status (open / resolved / false_positive), detected_at, alerted_at
scan_runs	id, repo_id, started_at, finished_at, commits_scanned, findings_count, error
allowlist	id, repo_id, pattern or path, reason, created_at
settings	key, value (for example Discord channel, default thresholds)

The **fingerprint** is a salted hash of (repository, commit, file, secret type, secret value). It lets the system detect duplicates and recurrences without retaining the secret itself. An index on (repo_id, detected_at) supports the dashboard's history view.

10. Backend API and Dashboard

10.1 REST endpoints

Method and path	Purpose
POST /repos	Register a repository or username to monitor
GET /repos	List monitored repositories with last scan status
DELETE /repos/{id}	Stop monitoring a repository
GET /findings	Paginated detection history with filters (severity, type, repo, status)
PATCH /findings/{id}	Mark a finding as resolved or as a false positive
GET /stats	Counts by severity and type, and findings per day for charts
POST /scan/{repo_id}	Trigger an immediate scan (useful during the demo)
POST /webhook/github	Optional receiver for push events (signature verified)

10.2 Dashboard views

- **Overview:** cards for total repositories, open findings and critical findings, plus a timeline chart and a breakdown by secret type.
- **Findings:** a table with severity badges, masked values, file and commit links, and resolve / false-positive actions.
- **Repositories:** add or remove monitored repositories, with last scan time and status.
- **Club view (extension):** aggregated exposure across many student repositories for faculty or club leads.

11. Security and Privacy Considerations

- **No plaintext secrets:** store only masked previews and fingerprints; do not log request bodies that contain diffs.
- **Credential hygiene:** the GitHub token and Discord webhook URL live in environment variables and are excluded from version control; the project's own repository should be scanned by the tool before it is published.
- **Least privilege:** the GitHub token needs read access to public data only.
- **Input handling:** repository names are validated, and diff content is treated as untrusted text and escaped before display to avoid cross-site scripting on the dashboard.
- **Access control:** dashboard login with hashed passwords, and rate limiting on the API.
- **Responsible use:** the system monitors only repositories that users register, and it is intended for notifying owners, not for harvesting credentials.

12. Technology Stack and 24-Hour Implementation Plan

Layer	Choice	Reason
Backend	Python, FastAPI, background tasks	Team already experienced; simple async support
Database	PostgreSQL with SQLAlchemy and Alembic	Familiar; reliable history queries
HTTP client	httpx or requests	GitHub API calls
Frontend	React with a charting library	Fast dashboard development
Alerts	Discord incoming webhook	Instant, free, easy to demo

Suggested project layout: `app/main.py`, `config.py`, `detector.py`, `poller.py`, `notifier.py`, `models.py`, `schemas.py` and `routers/` for repositories, findings and stats. The detector is written first because it is the core of the project and can be tested in isolation.

Phase	Hours	Deliverable
1. Setup	0 - 2	Repositories, environment files, database, test GitHub repo, Discord webhook
2. Detector	2 - 6	Regex rules, entropy function, masking, unit tests on sample strings
3. Poller and storage	6 - 10	Commit polling, diff parsing, findings saved in PostgreSQL
4. Alerts	10 - 12	Discord embeds with remediation text; end-to-end test commit
5. Dashboard	10 - 18	Overview, findings table, repository management (parallel with phases 3-4)
6. Polish	18 - 22	Severity, allowlist, false-positive control, charts, error handling
7. Demo and pitch	22 - 24	Rehearsed demo script, slides, backup recording

13. Evaluation Plan

A detector is only credible if it is measured. We propose a small, honest evaluation suitable for the hackathon.

- **Test corpus:** a set of fake but correctly formatted keys (never real ones) for each supported provider, plus negative examples such as UUIDs, commit hashes, base64 images, hashes in lock files and ordinary identifiers.
- **Metrics:** precision, recall and F1 on the corpus; false positives per 1,000 scanned lines on a few ordinary open-source repositories.
- **Latency:** time from push to alert, measured over repeated test commits; target below 30 seconds with a 10 to 15 second polling interval.
- **Robustness:** behaviour on large diffs, binary files, rate-limit responses and network failure.
- **Threshold tuning:** adjust the entropy thresholds and context lists to balance missed secrets against alert fatigue.

13.1 Demo scenario

- Show the dashboard empty, with one test repository registered.
- Push a commit containing three fake secrets of different types (cloud key, API key, database URL).
- Within seconds, show the Discord alerts with severity colours, masked values and remediation steps.
- Show the same findings appearing in the dashboard history and the charts.
- Push a commit that removes the key, and show that the history still lists the earlier finding, illustrating why revocation is required.
- Push a commit with only harmless random strings (UUIDs, hashes) to show that false-positive control works.

14. Limitations and Future Work

14.1 Limitations

- Detection quality depends on the pattern set; unknown formats are only caught by the weaker entropy heuristic.
- Entropy and keyword heuristics will produce some false positives and some misses.

- Polling introduces a delay of up to one polling interval and is bounded by API rate limits.
- The monitor cannot know whether a detected key is still active unless provider verification is added.
- Secrets that exist only in older history before monitoring started are not covered by the MVP.
- Because GitHub and other tools already scan public repositories, our value lies in student-oriented delivery and guidance rather than in detection coverage.

14.2 Future work

- Historical scan of the full commit history when a repository is first added.
- Optional verification of detected keys against provider APIs (with care and consent).
- A pre-commit hook or command-line scanner that reuses the same detection engine for prevention before a push.
- Support for GitLab and Bitbucket; scanning of pull-request comments and issues.
- Email notification to the commit author, and a scheduled weekly summary for clubs and faculty.
- Automated opening of an issue on the affected repository with remediation steps.

15. Conclusion

Credential leakage by students is a common and costly problem whose impact depends largely on how quickly the owner is informed. Established tools already detect secrets well, but they are not tailored to students or small communities. The proposed Secret Leakage Monitor combines proven detection techniques, namely provider-specific patterns and Shannon entropy, with a delivery layer designed for students: instant Discord or Slack alerts, plain-language remediation, masked storage and a simple dashboard. The design is deliberately scoped so that a small team with web and backend experience can build a reliable, demonstrable version within 24 hours, and it leaves clear paths for extension such as history scanning, verification and prevention hooks.

References

- [1] GitHub Docs, "About secret scanning alerts." <https://docs.github.com/en/code-security/concepts/secret-security/about-alerts>
- [2] GitHub Docs, "Secret scanning detection scope" (including public monitoring). <https://docs.github.com/en/code-security/reference/secret-security/secret-scanning-scope>
- [3] GitGuardian, product documentation and public monitoring overview. <https://www.gitguardian.com>
- [4] Truffle Security, TruffleHog (open-source secret scanner). <https://github.com/trufflesecurity/trufflehog>
- [5] Gitleaks (open-source secret scanner). <https://github.com/gitleaks/gitleaks>
- [6] Yelp, detect-secrets. <https://github.com/Yelp/detect-secrets>
- [7] C. E. Shannon, "A Mathematical Theory of Communication," Bell System Technical Journal, vol. 27, 1948.
- [8] GitHub Docs, REST API: commits, events and rate limits. <https://docs.github.com/en/rest>
- [9] Discord Developer Documentation, Webhooks. <https://discord.com/developers/docs/resources/webhook>

Note: secret formats and API limits change over time; verify each pattern and limit against the provider's current documentation before relying on it.